

Kiva Loans: Full-Dataset EDA (1.45M loans)

Kiva is a nonprofit lending platform: everyday people ("lenders") each contribute a small amount toward a loan for a borrower somewhere in the world, usually to grow a small business or cover a household need. Once enough lenders chip in, the loan is fully funded and the money is disbursed.

This analysis asks: does **how a loan's story is written** line up with **how quickly it gets funded**? Specifically, whether leaning on family/communal appeals, competence/independence framing, or urgency language correlates with faster funding - and whether that answer changes depending on the economic climate or the type of loan. A loan that sits unfunded longer is a worse experience for the borrower waiting on it and a worse use of a lender's attention, so a real, consistent link between narrative framing and funding speed would be an **actionable lever** - the kind of finding that could shape guidance for how loan write-ups get coached. If framing turns out to barely matter next to more structural factors, that's just as useful to know.

This notebook covers the descriptive groundwork across the complete, real dataset (not the 100-row illustrative sample the original proposal was built on); `2_full_dataset_modeling.ipynb` builds on it with statistical models that weigh every factor at once.

1. Setup

```
import re
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

SEED = 42
MIN_REGION_OBSERVATIONS = 10
MIN_SECTOR_OBSERVATIONS = 1000

KAGGLE_DATA_DIR = Path("/kaggle/input/datasets/tuannm3812/kiva-loans-hackathon-data")
if not KAGGLE_DATA_DIR.exists():
    KAGGLE_DATA_DIR = Path("/kaggle/input/kiva-loans-hackathon-data")

if KAGGLE_DATA_DIR.exists():
    DATA_PATH = KAGGLE_DATA_DIR / "Kiva_Loans.pkl"
else:

    def _find_project_root(start: Path) -> Path:
        candidate = start
        for _ in range(5):
            if (candidate / "data" / "Kiva_Loans_Sample.pkl").exists():
                return candidate
            if candidate.parent == candidate:
                break
            candidate = candidate.parent
        return start
```

```

try:
    PROJECT_ROOT = Path(__file__).resolve().parents[1]
except NameError:
    PROJECT_ROOT = _find_project_root(Path.cwd())
DATA_PATH = PROJECT_ROOT / "data" / "Kiva_Loans.pkl"

sns.set_theme(style="whitegrid", palette="viridis")
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["font.size"] = 12

```

2. Load Data

```

# The pickle is a list of row dicts, not a directly-pickled DataFrame
# (pd.read_pickle would raise) - a plain stdlib pickle.load handles both
# shapes, no custom package needed.
import pickle # noqa: E402

with open(DATA_PATH, "rb") as handle:
    _raw = pickle.load(handle)
df = pd.DataFrame(_raw) if isinstance(_raw, list) else _raw
print(f"Shape: {df.shape[0]:,} loans x {df.shape[1]} raw columns")

```

Shape: 1,453,846 loans x 27 raw columns

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1453846 entries, 0 to 1453845
Data columns (total 27 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   id                    1453846 non-null int64
 1   status                1453846 non-null object
 2   borrowerCount         1453846 non-null int64
 3   name                  1453846 non-null object
 4   gender                1453846 non-null object
 5   loanAmount            1453846 non-null float64
 6   lenderRepaymentTerm   1453846 non-null int64
 7   repaymentInterval     1453846 non-null object
 8   sector                1453846 non-null object
 9   activity              1453846 non-null object
10   use                   1453846 non-null object
11   city                  1392239 non-null object
12   latitude              1362860 non-null float64
13   longitude             1362860 non-null float64
14   country_iso           1453846 non-null object
15   country_name          1453846 non-null object
16   region                1453846 non-null object
17   country_ppp           1453846 non-null float64
18   fundsLentInCountry    1453846 non-null int64
19   country_latitude      1453846 non-null float64
20   country_longitude     1453846 non-null float64

```

```

21 description      1453846 non-null object
22 whySpecial       1414768 non-null object
23 image_url        1453846 non-null object
24 disbursalDate    1453846 non-null object
25 fundraisingDate  1453846 non-null object
26 raisedDate       1453846 non-null object
dtypes: float64(6), int64(4), object(17)
memory usage: 299.5+ MB

```

27 raw fields, almost all fully populated - most columns show close to 1,453,846 non-null values, with only a handful of geographic fields (latitude/longitude) carrying meaningful missingness. That's a clean, near-complete dataset to build on.

```

# A real sample of rows, restricted to columns that describe the *loan*
# rather than the *borrower* - deliberately excludes name/id/image_url
# and free-text/exact-timestamp fields that can still identify a real
# person even without a name column (the raw description usually opens
# with the borrower's name and a short biography; an exact date is
# specific enough to cross-reference a real loan on Kiva's own site).
preview_cols = [
    "gender", "borrowerCount", "loanAmount", "sector", "activity",
    "region", "country_name", "repaymentInterval",
]
df[preview_cols].head(8)

```

```

.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}

```

Each row is one real loan, with the attributes you'd expect: who it's for (**gender**, **borrower count**), how much (**loan amount**, in USD), what it's for (**sector**, **activity**), where (**region**, **country**), and the repayment structure. Two more field groups exist in the full data but aren't shown row-by-row above, for the privacy reason noted in the code: a free-text description written for lenders, and the two dates - **posted** and **fully funded** - whose gap is this entire notebook's subject.

3. Target Variable

Every loan has a posted date and a fully-funded date. The gap between them, `funding_speed_days`, is how many days it took lenders to fully fund the loan - close to 0 for a same-day loan, 21 for one that took three weeks. A negative or missing value means the record is unusable and is dropped, never guessed at. `funded_within_24h` is a simpler yes/no version of the same measure. **Both only exist for loans that did eventually get funded** - this data can't speak to whether a loan gets funded at all, only how fast it did once it succeeded.

```

fundraising = pd.to_datetime(df["fundraisingDate"], errors="coerce", utc=True)
raised = pd.to_datetime(df["raisedDate"], errors="coerce", utc=True)
df["funding_speed_days"] = (raised - fundraising).dt.total_seconds() / 86400

```

```

df["log_funding_speed"] = np.log1p(df["funding_speed_days"].clip(lower=0))
df["funded_within_24h"] = (df["funding_speed_days"] <= 1).astype("Int64")

year = fundraising.dt.year
df["analysis_period"] = pd.cut(
    year, bins=[-np.inf, 2019, 2021, np.inf],
    labels=["pre_pandemic", "pandemic_disruption", "post_pandemic"],
)

valid = df.loc[df["funding_speed_days"].notna() & (df["funding_speed_days"] >=
0)].copy()

print(f"Rows loaded: {len(df)}")
print(f"Rows with a valid completed outcome: {len(valid)}")
print(f"Rows excluded: {len(df) - len(valid)}")
print(f"Status among valid rows:\n{valid['status'].value_counts().to_string()}")

```

```

Rows loaded: 1453846
Rows with a valid completed outcome: 1453840
Rows excluded: 6
Status among valid rows:
status
funded      1452203
refunded      1637

```

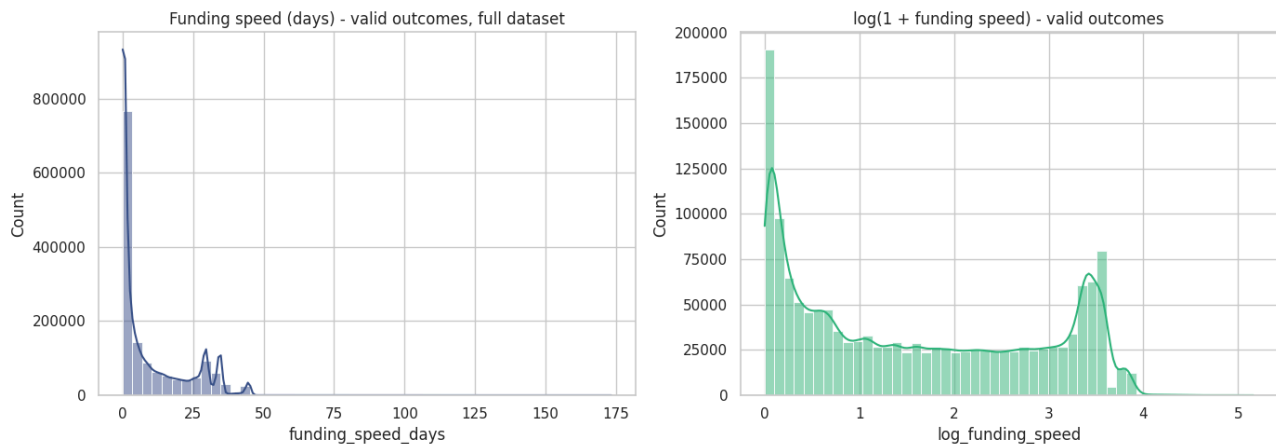
- The dataset is **almost entirely usable**: 1,453,840 of 1,453,846 loans (99.9996%) have a valid, non-negative funding duration - only 6 rows are dropped, for a data-quality reason (the funded date preceding the posted date), not because they were ignored.
- Among valid rows, **1,452,203 loans (99.89%) are funded** and 1,637 (0.11%) are refunded - refunded loans are kept on the same footing as funded ones, since a refund is a later, unrelated event; the loan still did get fully funded, which is what this notebook measures.
- With **14,500x more rows** than the proposal-week sample, this near-total coverage means the findings below aren't an artifact of small-sample noise.

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.histplot(valid["funding_speed_days"], kde=True, bins=50,
color=plt.cm.viridis(0.25), ax=axes[0])
axes[0].set_title("Funding speed (days) - valid outcomes, full dataset")

sns.histplot(valid["log_funding_speed"], kde=True, bins=50,
color=plt.cm.viridis(0.65), ax=axes[1])
axes[1].set_title("log(1 + funding speed) - valid outcomes")
plt.tight_layout()
plt.show()

```



Most loans fund very quickly (the tall bar near 0 on the left), with a long tail of slower loans stretching out for weeks. The right chart applies a **log transform**, which pulls that long tail in so the distribution looks closer to a symmetric bell shape - modeling techniques (used in `2_full_dataset_modeling.ipynb`) tend to perform better on data shaped like the right chart than the heavily skewed shape on the left; both charts describe the same underlying pattern.

4. Categorical Trends

How does funding speed differ across two simple categorical splits: **when** the loan was posted, and the borrower's **gender**?

```
period_counts = df["analysis_period"].value_counts(dropna=False).sort_index()
print("Rows per analysis period:")
print(period_counts.to_string())
```

```
within_24h_by_period = valid.dropna(subset=["funded_within_24h"]).groupby(
    "analysis_period", observed=True
)["funded_within_24h"].mean()
print("\nShare funded within 24 hours, by period:")
print(within_24h_by_period.to_string())
```

```
Rows per analysis period:
analysis_period
pre_pandemic      589823
pandemic_disruption  298549
post_pandemic     565474
```

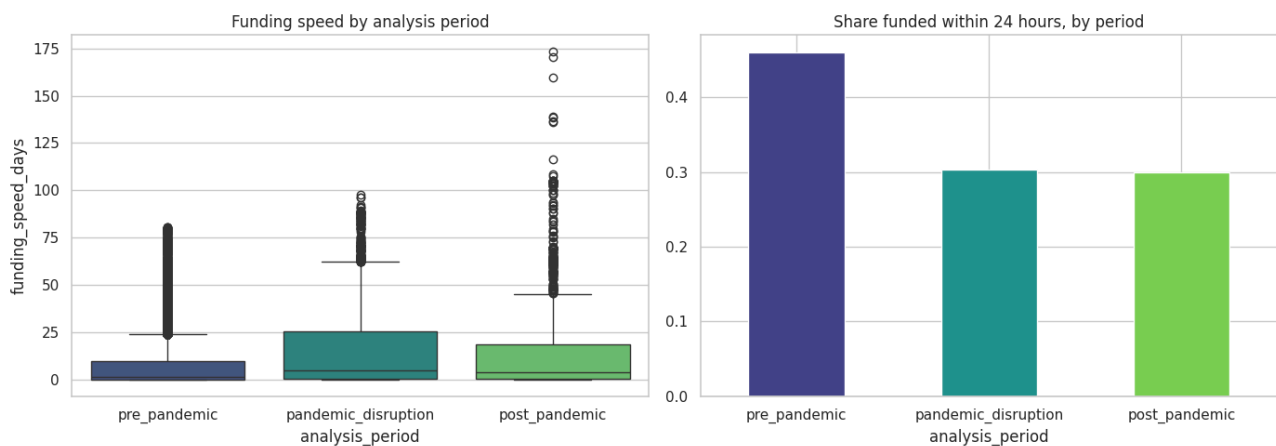
```
Share funded within 24 hours, by period:
analysis_period
pre_pandemic      0.46028
pandemic_disruption  0.30321
post_pandemic     0.299945
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.boxplot(
    data=valid, x="analysis_period", y="funding_speed_days",
    hue="analysis_period", legend=False, palette="viridis", ax=axes[0],
)
axes[0].set_title("Funding speed by analysis period")
```

```

within_24h_by_period.astype(float).plot(
    kind="bar", color=plt.cm.viridis(np.linspace(0.2, 0.8,
len(within_24h_by_period))), ax=axes[1]
)
axes[1].set_title("Share funded within 24 hours, by period")
axes[1].tick_params(axis="x", rotation=0)
plt.tight_layout()
plt.show()

```



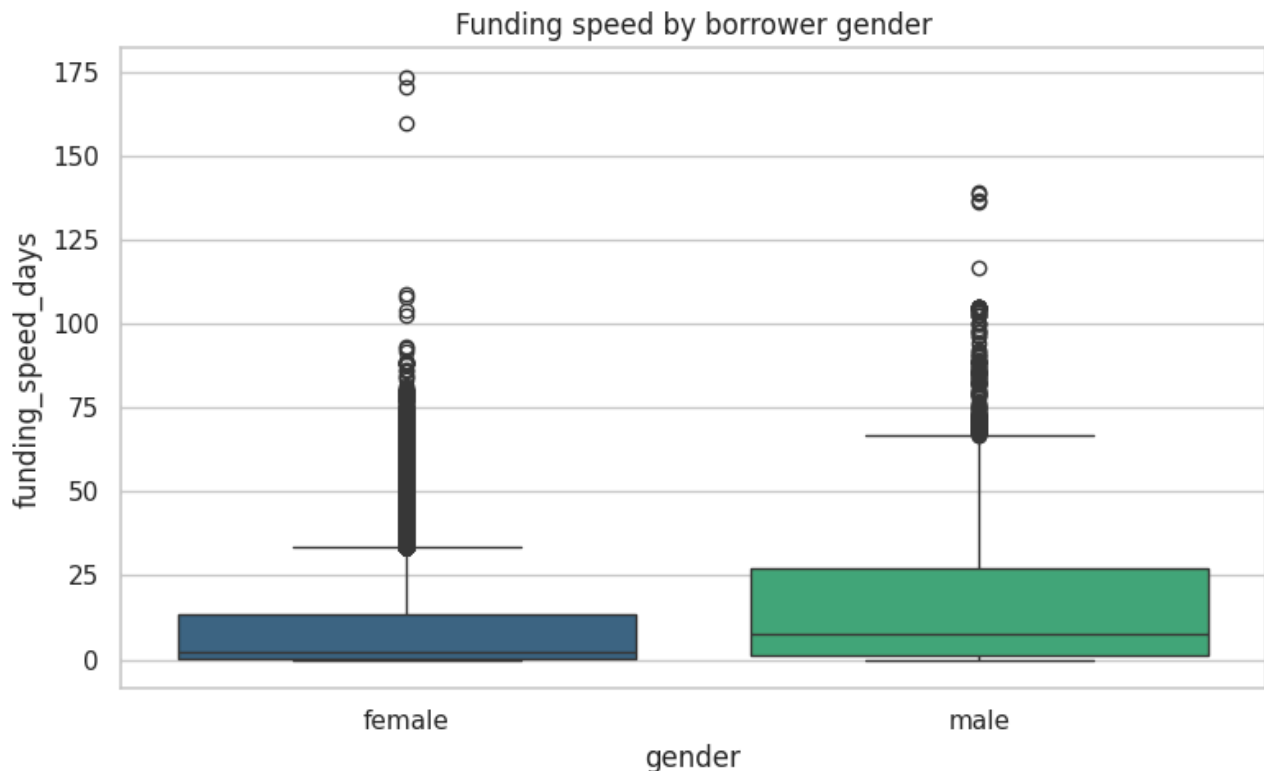
The headline finding of this notebook: the share of loans funded within 24 hours nearly halved and has not recovered through the end of the data (2025) - **46.0% pre-pandemic → 30.3% during pandemic disruption → 30.0% post-pandemic**. Before 2020, almost half of all loans were fully funded within a day of posting; since 2020, that's dropped to under a third, and it has stayed there for the four years of data after the disruption itself ended. This isn't a small blip either - 589,823 loans fall in the "before" group and 565,474 in the "after" group, so it isn't a handful of unusual loans skewing the picture. Whatever changed about how this marketplace funds loans around 2020 has persisted to date - this data can describe that persistence, not explain it, and cannot say whether it will ever reverse.

```

fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(data=valid, x="gender", y="funding_speed_days", hue="gender",
legend=False, palette="viridis", ax=ax)
ax.set_title("Funding speed by borrower gender")
plt.tight_layout()
plt.show()

print("Median funding speed (days), by gender:")
print(valid.groupby("gender",
observed=True)["funding_speed_days"].median().to_string())

```



```
Median funding speed (days), by gender:
gender
female    2.332182
male      7.713970
```

Loans posted under a **male** borrower take noticeably longer to fund than loans posted under a **female** borrower, even in this simple, one-factor view. This is worth keeping in mind heading into the modeling notebook, where the gap survives controlling for everything else about the loan - smaller than the biggest sector and region effects, but larger than the loan amount itself, and larger than every narrative-framing term combined. This chart is the first, simplest hint of that result.

5. Categorical Features

A wider sweep across the remaining categorical fields: **sector** (what the loan is for), **region**, and **repayment interval** (how the borrower repays). Sector and region are collapsed first so that any category with too few loans to trust - fewer than `MIN_SECTOR_OBSERVATIONS` (1,000) for sector, `MIN_REGION_OBSERVATIONS` (10) for region - is folded into "Other" rather than charted on its own; the same rule the modeling notebook uses, so a single thin, noisy category can't produce a misleadingly dramatic bar. Every chart also labels each bar with its loan count (`n=`), so a fast- or slow-funding category can be checked against how much data actually backs it, not just how the bar looks.

```
overall_avg_speed = valid["funding_speed_days"].mean()

def _barh_avg_speed_with_counts(series_grouped_by: pd.Series, ax, title: str,
                                show_legend: bool = False) -> None:
```

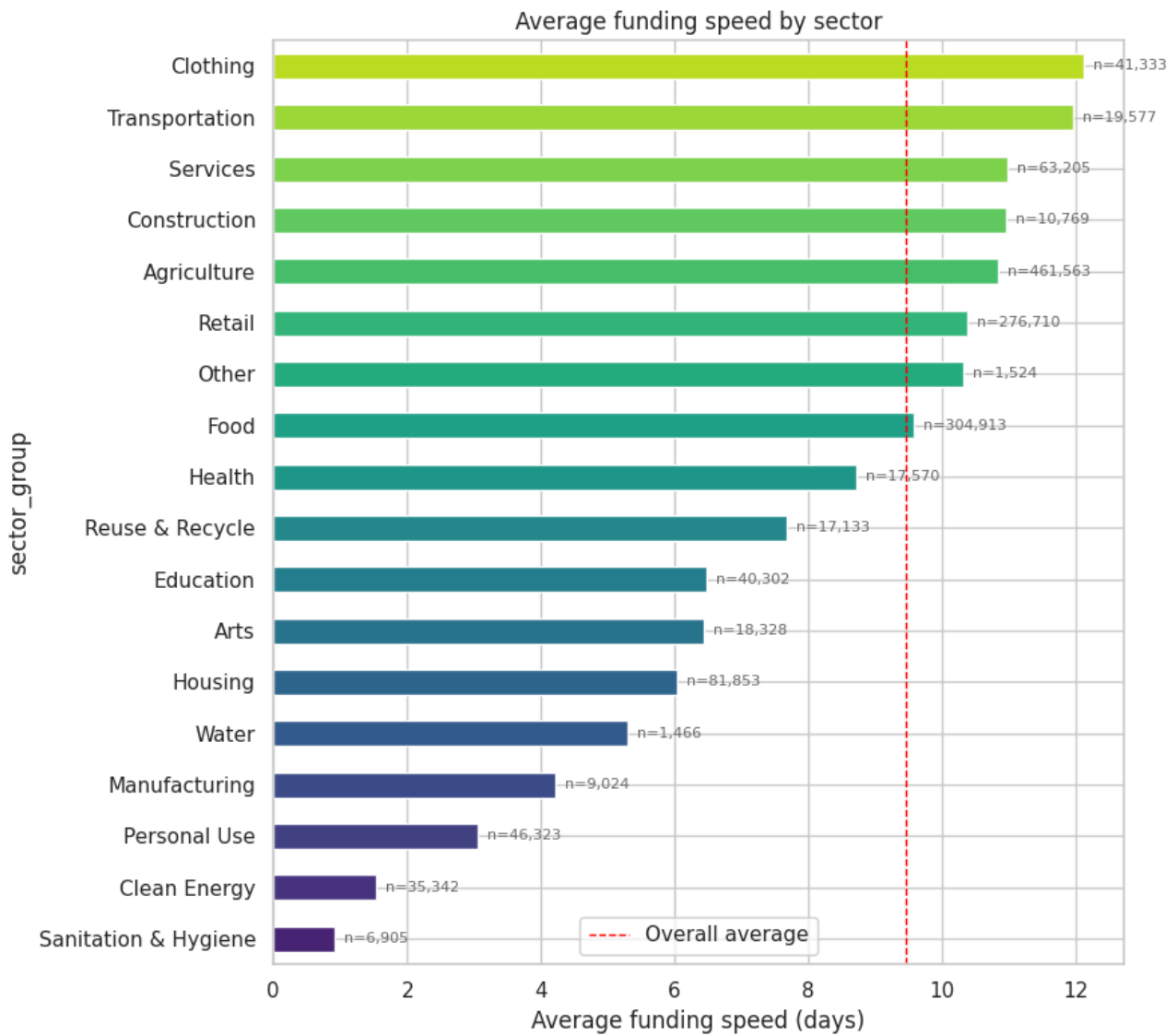
```

    """Horizontal bar chart of mean funding speed per category, each bar labeled with
    its loan count."""
    stats = valid.groupby(series_grouped_by, observed=True)
    ["funding_speed_days"].agg(["mean", "count"]).sort_values("mean")
    stats["mean"].plot(kind="barh", color=plt.cm.viridis(np.linspace(0.1, 0.9,
len(stats))), legend=False, ax=ax)
    avg_line = ax.axvline(overall_avg_speed, color="red", linestyle="--",
linewidth=1, label="Overall average")
    for i, (mean_val, count_val) in enumerate(zip(stats["mean"], stats["count"])):
        ax.text(mean_val, i, f" n={count_val:},", va="center", fontsize=8,
color="dimgray")
    ax.set_xlabel("Average funding speed (days)")
    ax.set_title(title)
    if show_legend:
        ax.legend(handles=[avg_line])

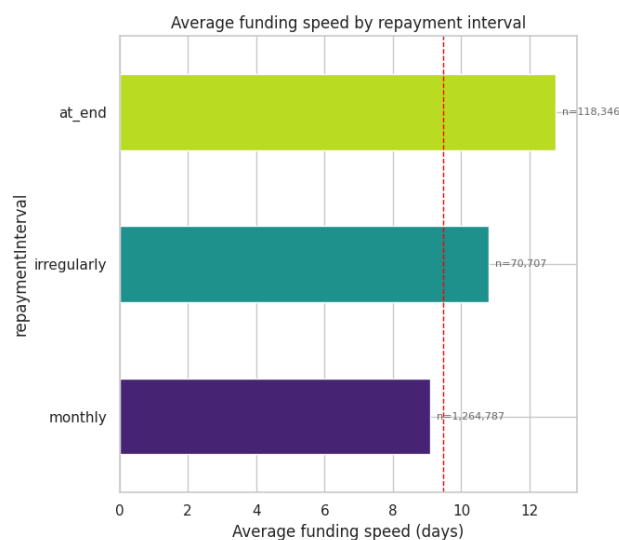
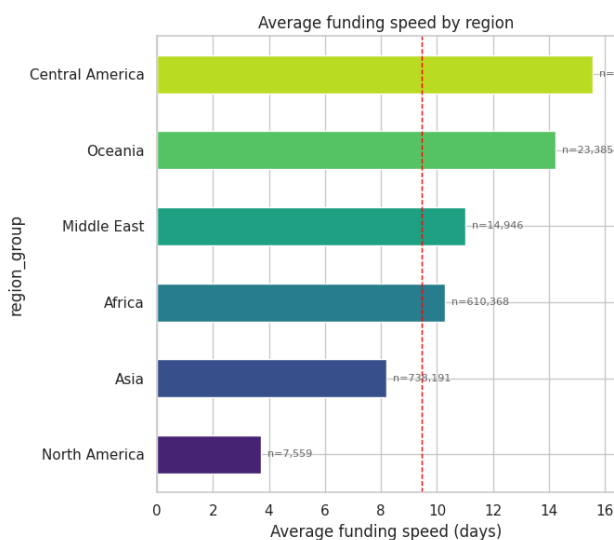
for col, min_obs, new_col in [("sector", MIN_SECTOR_OBSERVATIONS, "sector_group"),
("region", MIN_REGION_OBSERVATIONS, "region_group")]:
    counts = valid[col].value_counts()
    major = counts[counts >= min_obs].index
    valid[new_col] = valid[col].where(valid[col].isin(major), "Other")

fig, ax = plt.subplots(figsize=(9, 8))
_barh_avg_speed_with_counts(valid["sector_group"], ax, "Average funding speed by
sector", show_legend=True)
plt.tight_layout()
plt.show()

```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
_barh_avg_speed_with_counts(valid["region_group"], axes[0], "Average funding speed by
region")
_barh_avg_speed_with_counts(valid["repaymentInterval"], axes[1], "Average funding
speed by repayment interval")
plt.tight_layout()
plt.show()
```



- **Sector matters enormously** - the gap between the fastest- and slowest-funding sectors dwarfs anything narrative framing produces on its own, and every sector shown is backed by well over a thousand loans, so the gap isn't a thin-sample artifact. Directly foreshadows the modeling notebook's sector findings.
- **Region shows a similarly wide spread** - funding speed varies substantially by where the borrower is located, independent of anything about how the loan is written.
- **Repayment interval shows a clear pattern too** - loans repaid as a single lump sum at the end of the term take noticeably longer to fund than loans repaid monthly or irregularly.

None of these are narrative-framing signals - they're all about the loan itself. Seeing this much variation from structural fields alone, before any modeling, previews the modeling notebook's central finding: structure outweighs narrative framing.

6. Narrative Framing

Each loan description is scored on three persuasion styles, each grounded in research on what makes an ask persuasive:

- **Family/communal** - mentions of children, family roles (mother, father, spouse) - a "this affects real people you can relate to" appeal.
- **Agency/competence** - mentions of deciding, managing, running a business independently - signals capability rather than need.
- **Urgency** - explicit urgency/emergency language ("today," "right now," "before it's too late") - a time-pressure appeal.

Each style is counted as a **rate per 100 words** rather than a raw count, so a longer description doesn't automatically score as "more urgent" just because it has more words.

```
FAMILY_PATTERN = re.compile(r"\b(child|children|family|son|daughter|mother|father|
wife|husband|school)\b", re.I)
AGENCY_PATTERN = re.compile(r"\b(decide|plan|manage|responsible|hard.?working|
independent|own|run|lead)\w*\b", re.I)
URGENCY_PATTERN = re.compile(r"\b(urgent|immediately|emergency|crisis|desperate|asap|
quickly)\w*\b", re.I)
```

Raw descriptions carry stray HTML tags (mostly `
>` line breaks) -

```

# stripped here, in `valid` itself, so every later use of `description`
# (framing rates, sentiment, topic modeling) sees clean text instead of
# tags showing up as spurious "words" (e.g. "br").
valid["description"] = (
    valid["description"].fillna("")
    .str.replace(r"<[^>]+>", " ", regex=True)
    .str.replace(r"\s+", " ", regex=True)
    .str.strip()
)
description = valid["description"]
word_count = description.str.split().str.len().clip(lower=1)

def _rate_per_100_words(pattern: re.Pattern, text: pd.Series, words: pd.Series) ->
pd.Series:
    return text.str.count(pattern) / words * 100

valid["family_mentions_per_100_words"] = _rate_per_100_words(FAMILY_PATTERN,
description, word_count)
valid["agency_mentions_per_100_words"] = _rate_per_100_words(AGENCY_PATTERN,
description, word_count)
valid["urgency_mentions_per_100_words"] = _rate_per_100_words(URGENCY_PATTERN,
description, word_count)

```

7. Sentiment Analysis

Alongside framing, each description also gets a sentiment score from **VADER**, a well-established, off-the-shelf tool that reads text and returns a single score from -1 (very negative) to +1 (very positive) - the same idea as a star rating summarizing a review. Scored on a random sample of 20,000 descriptions for speed.

```

import nltk # noqa: E402

try:
    nltk.data.find("sentiment/vader_lexicon.zip")
except LookupError:
    nltk.download("vader_lexicon", quiet=True)

from nltk.sentiment.vader import SentimentIntensityAnalyzer # noqa: E402

analyzer = SentimentIntensityAnalyzer()
sentiment_sample = valid.sample(min(20_000, len(valid)), random_state=SEED).copy()
sentiment_sample["sentiment_compound"]
= sentiment_sample["description"].fillna("").apply(
    lambda text: analyzer.polarity_scores(text)["compound"]
)

print("Sentiment (compound score) summary, 20K-row sample:")
print(sentiment_sample["sentiment_compound"].describe().to_string())

```

```

Sentiment (compound score) summary, 20K-row sample:
count    20000.000000

```

```

mean      0.779637
std       0.286086
min       -0.929000
25%       0.743000
50%       0.886000
75%       0.954500
max       0.998600

```

Kiva loan descriptions are **overwhelmingly positive in tone**: the average score is 0.78 out of a possible 1.0, and the typical (median) description scores an even higher 0.89; even the more modestly-toned quarter of descriptions still scores a strongly positive 0.74. Almost every description on this platform is written in an upbeat, hopeful voice, with very little genuinely neutral or negative writing. That's a **ceiling effect** worth noting: when nearly everything already sits near the top of the scale, sentiment has little room left to vary from loan to loan, which matters when interpreting the sentiment result in the modeling notebook.

8. Topic Modeling

The three framing scores above only count words from a fixed, hand-picked list. **Topic modeling** takes the opposite approach: it reads the descriptions and lets common word-groupings emerge on their own, with no predefined list. Specifically, this uses **TF-IDF + NMF** - TF-IDF scores each word by how distinctive it is to a description (common words like "the" score low; distinctive words like "tailor" or "livestock" score high), and NMF groups descriptions that share distinctive words into a fixed number of topics. Run on the same 20,000-description sample as the sentiment analysis above, for the same speed reason.

```

from sklearn.decomposition import NMF # noqa: E402
from sklearn.feature_extraction.text import TfidfVectorizer # noqa: E402

N_TOPICS = 8
N_TOP_WORDS = 8

vectorizer = TfidfVectorizer(max_features=1000, stop_words="english", min_df=5)
tfidf_matrix = vectorizer.fit_transform(sentiment_sample["description"].fillna(""))

nmf_model = NMF(n_components=N_TOPICS, random_state=SEED, max_iter=300)
topic_weights = nmf_model.fit_transform(tfidf_matrix)
sentiment_sample["dominant_topic"] = topic_weights.argmax(axis=1)

feature_names = vectorizer.get_feature_names_out()
topic_labels = {}
for topic_idx, topic in enumerate(nmf_model.components_):
    top_words = [feature_names[i] for i in topic.argsort()[-N_TOP_WORDS:][::-1]]
    topic_labels[topic_idx] = ", ".join(top_words)
    print(f"Topic {topic_idx}: {' '.join(top_words)}")

```

```

Topic 0: pigs, pig, raise, supplies, nwtf, fattening, years, raising
Topic 1: business, loan, income, children, years, family, kiva, old
Topic 2: sanitary, toilet, risks, hazard, health, using, aware, reducing
Topic 3: water, village, filter, clean, drinking, boiling, fuel, safeguard
Topic 4: store, general, items, canned, goods, like, personal, products
Topic 5: farm, farming, inputs, quality, farmer, high, seeds, smallholder

```

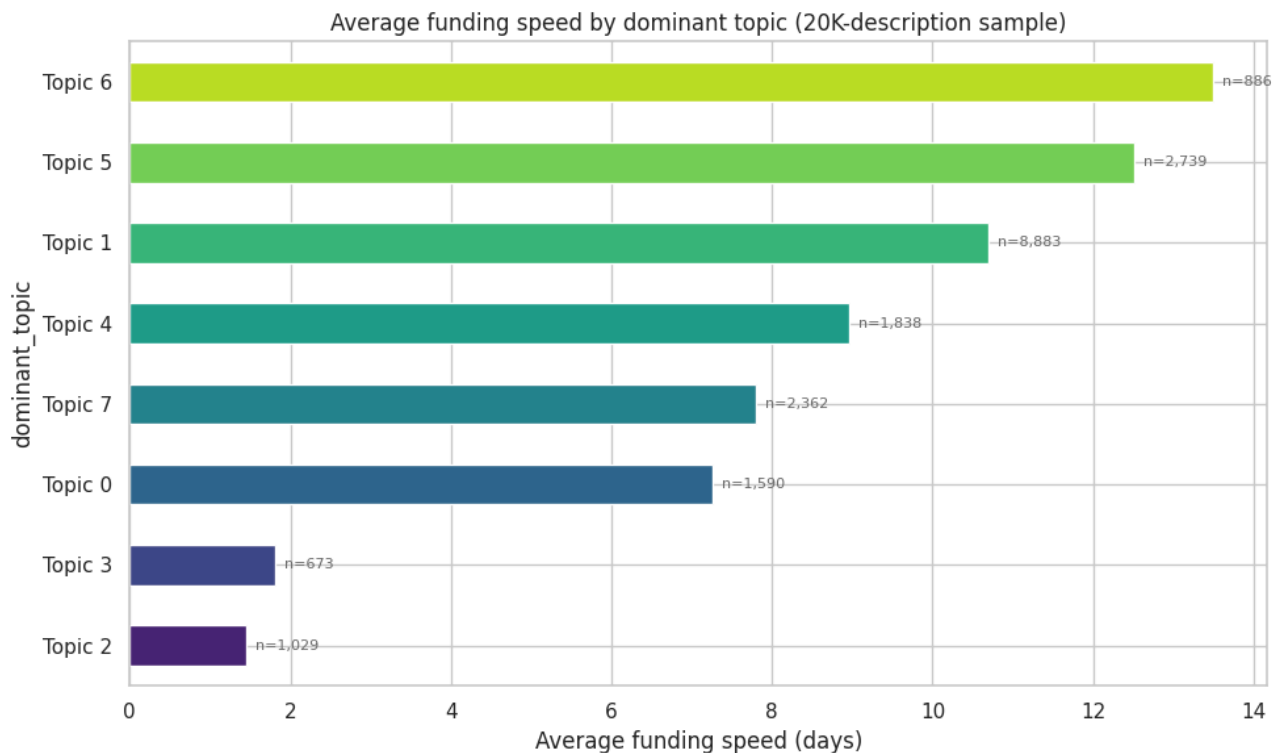
Topic 6: group, solar, acre, total, fund, farmers, light, acres

Topic 7: business, nwtf, philippines, php, entrepreneur, fish, additional, 000

```
topic_speed = sentiment_sample.groupby("dominant_topic")
["funding_speed_days"].agg(["mean", "count"]).sort_values("mean")
topic_speed["top_words"] = [topic_labels[i] for i in topic_speed.index]
print(topic_speed.to_string())

fig, ax = plt.subplots(figsize=(10, 6))
topic_speed["mean"].plot(
    kind="barh", color=plt.cm.viridis(np.linspace(0.1, 0.9, len(topic_speed))),
    ax=ax,
)
for i, (mean_val, count_val) in enumerate(zip(topic_speed["mean"],
topic_speed["count"])):
    ax.text(mean_val, i, f" n={count_val:,"}, va="center", fontsize=8,
color="dimgray")
ax.set_yticks(range(len(topic_speed)))
ax.set_yticklabels([f"Topic {i}" for i in topic_speed.index])
ax.set_xlabel("Average funding speed (days)")
ax.set_title("Average funding speed by dominant topic (20K-description sample)")
plt.tight_layout()
plt.show()
```

	mean	count	
top_words			
dominant_topic			
2	1.455901	1029	sanitary, toilet, risks, hazard, health,
using, aware, reducing			
3	1.814373	673	water, village, filter, clean, drinking,
boiling, fuel, safeguard			
0	7.258376	1590	pigs, pig, raise, supplies, nwtf,
fattening, years, raising			
7	7.803372	2362	business, nwtf, philippines, php, entrepreneur,
fish, additional, 000			
4	8.964443	1838	store, general, items, canned, goods, like,
personal, products			
1	10.700803	8883	business, loan, income, children, years,
family, kiva, old			
5	12.513396	2739	farm, farming, inputs, quality, farmer, high,
seeds, smallholder			
6	13.492799	886	group, solar, acre, total, fund,
farmers, light, acres			



Insight: the topics that emerge line up with recognizable, real themes rather than random word clusters - livestock (pigs), sanitary and health, clean water access, general stores, farming, and a solar/group-lending topic, alongside a couple of broad business/livelihood topics. That's confirmation the descriptions have genuine thematic structure beyond the three hand-picked framing categories tested earlier.

The funding-speed gap between topics is large - the sanitary/health topic funds in **1.5 days on average**, over nine times faster than the solar/group-lending topic's **13.5 days**. That's a bigger swing than any single narrative-framing signal produces, and every topic above is backed by several hundred to several thousand loans (see the *n=* label on each bar) - not a handful of outliers driving the gap. But every topic here spans a mix of sectors and loan sizes, so **this is a starting point for a deeper dive, not a standalone conclusion** - a topic effect could still just be tracking which sectors happen to write about which subjects, the same caveat that applies to the framing correlations below.

9. Feature Correlations

A correlation is a score from -1 to +1 for whether two things tend to move together: 0 means no relationship, +1 means they consistently rise together, -1 means one falls as the other rises. This section checks the raw, one-at-a-time correlation between funding speed and each of the three framing styles above, the loan amount, and the repayment term (how long the borrower has to repay).

```
narrative_cols = ["family_mentions_per_100_words", "agency_mentions_per_100_words",
"urgency_mentions_per_100_words"]
valid["log_loan_amount"] = np.log1p(valid["loanAmount"])
structural_cols = ["log_loan_amount", "lenderRepaymentTerm"]

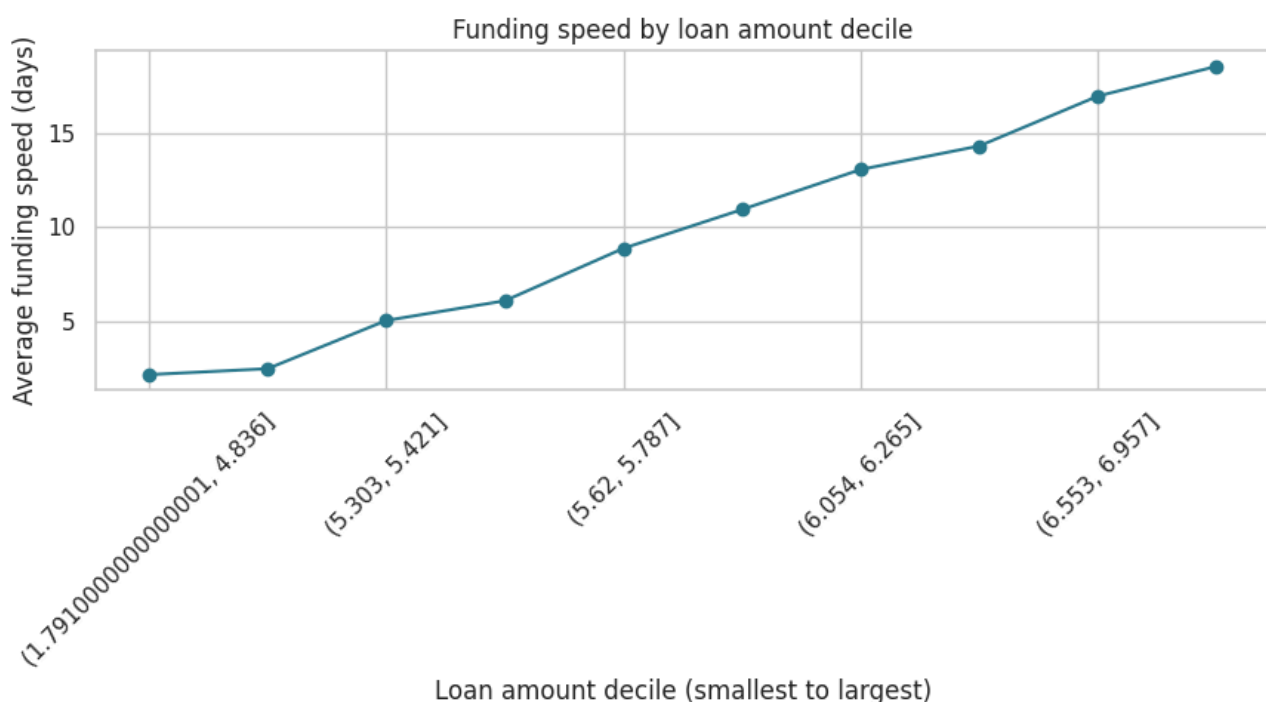
corr_table = valid[narrative_cols + structural_cols + ["funding_speed_days"]].corr()
```

```
["funding_speed_days"].drop("funding_speed_days")
print("Correlation with funding speed (days):")
print(corr_table.sort_values().to_string())
```

```
Correlation with funding speed (days):
family_mentions_per_100_words    -0.018791
urgency_mentions_per_100_words    0.010423
agency_mentions_per_100_words     0.058013
lenderRepaymentTerm               0.284895
log_loan_amount                   0.429447
```

```
loan_amount_bin = pd.qcut(valid["log_loan_amount"], 10, duplicates="drop")
speed_by_loan_amount_bin = valid.groupby(loan_amount_bin, observed=True)
["funding_speed_days"].mean()

fig, ax = plt.subplots(figsize=(9, 5))
speed_by_loan_amount_bin.plot(kind="line", marker="o", color=plt.cm.viridis(0.4),
ax=ax)
ax.set_xlabel("Loan amount decile (smallest to largest)")
ax.set_ylabel("Average funding speed (days)")
ax.set_title("Funding speed by loan amount decile")
ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```



- **Loan structure dominates over narrative framing.** Larger loan amounts ($r = +0.43$) and longer repayment terms ($r = +0.28$) are the strongest correlates of slower funding - a bigger ask naturally takes longer to fill, and the decile chart above shows this relationship is close to monotonic: average funding speed rises steadily from the smallest to the largest loans.
- **The three framing styles barely register by comparison** - an order of magnitude weaker: family framing $r = -0.019$ (a whisper of a link to faster funding), urgency $r = +0.010$ (essentially

no relationship either way), agency framing $r = +0.058$ (a whisper of a link to slower funding - the opposite of what a naive "sound confident and it'll fund faster" assumption would predict).

This doesn't mean framing doesn't matter at all - a simple one-at-a-time correlation can't separate framing's own effect from other things that happen to travel together with it (e.g. larger loans might also just happen to be written in a different style). **Urgency framing turned out to be a cautionary tale about stopping the analysis too early.** Its raw correlation here is essentially zero ($r = +0.010$). Controlling for loan size, sector, and everything else at once (the modeling notebook's first-pass regression) made urgency look like a strong, precise association with faster funding - a classic "confound was masking a real effect" story. But a country-clustered sensitivity check - refitting with standard errors clustered by country instead of assuming every loan is independent - showed that apparent association doesn't hold up either (full detail in `2_full_dataset_modeling.ipynb` Section 7.1): its p-value rises from well under 0.001 to roughly 0.44, no longer distinguishable from no association. **Two rounds of "controlling for more" changed the answer twice** - first making urgency look important, then showing that importance doesn't survive a check under that coarser dependence assumption. A third round - testing the right quantity, not just testing it robustly (that notebook's Section 7.2) - narrowed it further still. What is left is a descriptive pattern, not a supported association: more family language goes with faster funding in **two pooled region categories** - Middle East (Palestine + Yemen) and Central America (Honduras + Nicaragua) - in the same direction in all three same-data model fits we ran. But each category is only two countries, and once the p-value is referred to a few-cluster t distribution instead of the usual normal approximation, neither clears significance. The model also estimates one pooled slope per category, never a slope for any individual country. A hypothesis for a country-stratified test, not a region-level or country-level rule. That is a far more specific, and more cautionary, finding than "timing, region, and loan size all matter," which is what an analysis stopping after the first round would have concluded. Sentiment tone's association is a genuinely open question rather than a third survivor - it holds up in this project's richer authoritative model but not in the modeling notebook's simpler one, a disagreement the modeling notebook reports directly rather than picking whichever result looks better.

10. Key Findings

10.1 Technical Interpretation

- The dataset is complete and clean - 1,453,840 of 1,453,846 loans (99.9996%) have a usable funding-speed outcome.
- Funding speed shifted to a persistently slower regime after 2019: the share funded within 24 hours fell from 46% to about 30% and has not recovered through the end of the data (2025) - persistence to date, not a claim it can never recover.
- Loan descriptions cluster tightly at the positive end of the sentiment scale (median 0.89/1.0), limiting how much sentiment alone can explain.
- Loan structure (amount, repayment term) correlates with funding speed an order of magnitude more strongly than any single narrative-framing signal; gender shows a visible gap even before controlling for anything else.
- A cluster-robust robustness check in the modeling notebook (Section 7.1) substantially revised the narrative-framing picture: urgency framing's apparent association, and most of family framing's conditional structure, don't survive standard errors clustered by country. Tested

with the correct within-region contrast (that notebook's Section 7.2), one descriptive pattern appears everywhere it's checked: family framing goes with faster funding in two pooled region categories - Middle East (Palestine + Yemen) and Central America (Honduras + Nicaragua) - but each is two countries, and under a few-cluster t reference neither is significant, so it is a hypothesis, not a statistically supported association. Elsewhere (Africa, Asia, North America, Oceania) nothing clears even the conventional test in any fit. Sentiment tone's association is a genuinely open question, not a confirmed survivor - it holds up in this project's richer authoritative model but not in the modeling notebook's own simpler one.

- Topic modeling on the loan descriptions surfaces coherent real-world themes (livestock, sanitary/health, clean water, farming, general retail); funding speed varies more than ninefold across topics (1.5 to 13.5 days on average) - the largest single gap found anywhere in this notebook, though it likely tracks sector/loan-type differences as much as writing style.

10.2 Business Impact

- **The pandemic-era slowdown is a durable, concrete finding** - something changed structurally about this marketplace around 2020 that a "things will bounce back" assumption doesn't hold. Worth investigating operationally, not just noting.
- **A ceiling effect in sentiment means "sound more positive" is unlikely to be worth coaching for** - almost every loan is already written in an upbeat voice, so there's little room left to differentiate on tone alone.
- **Loan size and repayment terms are the strongest structural predictors of speed by far** - they're not something a platform can change on an existing loan, but they're useful for setting realistic funding-time expectations, and they suggest more value in a structural review than in narrative-framing coaching alone.
- **What a loan is for and where it's from is linked to funding speed more strongly than how it's written** - the sector, region, and topic-modeling sweeps all show swings far larger than any narrative-framing signal. A platform-level "why do some loan types fund so much slower" review would likely pay off more than writing-style coaching alone.
- **Narrative framing's value is narrower than a first-pass model suggested, and it's worth stress-testing before acting on it** - the modeling notebook's robustness check shows urgency's apparent link to speed doesn't survive a country-clustered standard-error assumption, and neither does most of family framing's conditional structure. What remains after testing is not a writing rule at all: a descriptive pattern - family framing and faster funding in two pooled two-country categories (Middle East = Palestine + Yemen; Central America = Honduras + Nicaragua) - that is consistent in direction but not statistically supported once the two-country cluster base is taken seriously. So a "mention family" recommendation isn't supported anywhere in this dataset; the pattern is worth a country-stratified test, nothing more. Even sentiment tone's (counterintuitive) association, which looked like a second survivor, turned out to depend on exactly which model is fitted - a reminder to keep testing rather than stop at the first result that looks robust.